

АлтГТУ
ДИПЛОМНЫЙ ПРОЕКТ
группа 1ИСП-21
специальность СПО 09.02.07
«Информационные системы и
программирование»
Авхимович Артём Петрович
2026 г.

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет
им. И.И. Ползунова»
Университетский технологический колледж

УДК 004.42 ; 004.738.5

ДИПЛОМНЫЙ ПРОЕКТ

ДП 09.02.07.1

Разработка веб-приложения для заметок и отслеживания задач

Пояснительная записка

Студент группы 1ИСП-21

Авхимович Артём Петрович

Руководитель проекта

Преподаватель каф.ИСЭ

К.В. Воробьёв

Барнаул 2026

Реферат

Выпускная квалификационная работа состоит из введения, трёх глав, заключения, списка использованных источников литературы и приложения, выполнена на 41 листах машинописного текста, содержит 18 рисунков, 9 таблиц, 4 источника информации.

Объект исследования: процесс хранения и структурирования личной информации пользователя.

В рамках данной работы проводится исследовательская работа с анализом предметной области, сравнением существующих решений и проектированием веб-приложения для создания заметок и отслеживания задач. В ходе работы выявлены неудобства текущих способов хранения информации, а также необходимость создания системы, объединяющей заметки, задачи и связи между ними.

Внедрение такой системы позволит упростить работу пользователя с личной информацией, сократить время на поиск нужных материалов и повысить удобство отслеживания задач.

					<i>ДП 09.02.07.01.000 ПЗ</i>		
Изм.	Лист	№ документа	Подпись	Дата	<i>Разработка веб-приложения для заметок и отслеживания задач</i>		
Разраб.		Авхимович А.П.					
Провер.							
Реценз.							
Н.контр.							
Утверд.		Авдеев А. С.			Лит		
					Лист		
					Листов		
					У	2	41
					АлтГТУ, УТК, гр.1ИСП-21		

Содержание

Введение.....	3
Основная часть	5
Глава 1. Аналитическая часть	5
1.1 Анализ предметной области	5
1.2 Предпроектное обследование объекта.....	6
1.2.1 Анализ целевой аудитории	6
1.2.2 Анализ текущего процесса управления информацией.....	7
1.3 Анализ существующих решений	8
Глава 2. Проектная часть.....	11
2.1 Описание процесса разработки модели предмета (объекта) исследования.....	11
2.2 Требования к информационной системе.....	16
2.2.1 Функциональные требования.....	16
2.2.2 Нефункциональные требования	18
2.3 Проектирование базы данных.....	18
2.3.1 Выбор системы управления базами данных.....	18
2.3.2 Концептуальная модель данных.....	19
2.3.3 Логическая структура базы данных	20
2.3.4 Схема БД	22
2.4 Построение функционально-алгоритмической структуры.....	23
2.4.1 Алгоритм аутентификации и авторизации пользователя.....	23
2.4.2 Алгоритм создания и сохранения заметки	24
2.5 Проектирование пользовательского интерфейса	26
2.6 Выбор технологий	34
Глава 3. Реализация проекта	35
3.1 Описание реализации	35
3.2 Тестирование и отладка.....	35
3.3 Внедрение (при необходимости).....	35
Заключение	36
Список источников.....	37

Введение

В условиях повсеместной цифровизации и роста ежедневной входящей информации, пользователи сталкиваются с проблемой переизбытка информации. Их задачи и заметки обычно хранятся в разных местах, что часто приводит к потере контекста и снижению продуктивности. Несмотря на устойчивый рост рынка приложений, призванных повысить продуктивность, вопрос организации рабочего пространства при переключении между различными сервисами остаётся нерешённым.

Актуальность этой работы обоснована отсутствием на рынке удобного инструмента, который бы гармонично объединял ведение заметок и управление задачами в удобной для восприятия человеком форме. Существующие решения либо предлагают слишком простой интерфейс без возможностей для анализа (Google Keep, Apple Notes), либо, наоборот, требуют значительных усилий для освоения настроек и установки сторонних дополнений или платные функции облачного хранилища (Obsidian, Logseq), что существенно усложняет начало работы. Таким образом, существует потребность инструменте, который мог бы сочетать преимущества этих систем, одновременно устраняя их основные недостатки.

Платформа «Romni» создаётся как раз для ответа на эту потребность. Она объединяет принципы управления личными знаниями (Personal Knowledge Management, PKM) и функцию отслеживания задач в одном веб-приложении. Возможность видеть связи между заметками на интерактивном виде, а также интеграция сроков выполнения и статусов задач делает такой инструмент весьма востребованным. Он будет полезен студентам, исследователям и всем специалистам, кто работает с большими объёмами информации.

В результате выполнения работы, ожидается получить функционирующее веб-приложение с удобным взаимодействием с заметками и задачами.

					ДП.09.02.07.01.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		3

Основная часть работы включает три главы: аналитическую, в которой исследуется предметная область и проводится анализ существующих решений, проектную, в которой описывается моделирование процессов, проектирование базы данных и интерфейса, и главу реализации, где представлены результаты разработки и тестирования системы.

					ДП.09.02.07.01.000 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подп.	Дата		

Основная часть

Глава 1. Аналитическая часть

1.1 Анализ предметной области

Предметная область данного проекта охватывает сферу управления персональными знаниями (РКМ) и цифрового трекинга задач. В последние годы наблюдается устойчивый рост спроса на инструменты продуктивности, что обусловлено переходом к удалённым форматам обучения и работы, а также увеличением объёма неструктурированной информации, с которой ежедневно сталкиваются пользователи.

Согласно аналитическим отчётам, 54% организаций используют более пяти различных платформ для документирования и обмена информацией, что приводит к фрагментации рабочего пространства и снижению продуктивности [1]. Более 47% специалистов тратят от 1 до 5 часов в день на поиск конкретной информации в разрозненных источниках [1].

Рынок приложений для продуктивности демонстрирует устойчивый рост: в 2024 году его объём составил \$32,5 млрд, при этом основную долю занимают офисные пакеты и инструменты для работы с заметками [2]. Несмотря на обилие решений, проблема потери контекста при переключении между сервисами остаётся актуальной.

Ключевой проблемой данной отрасли является потеря контекста при работе с разрозненными данными. Пользователи часто фиксируют идеи в текстовых редакторах, но не имеют возможности быстро восстановить логические связи между ними. Существующие решения либо предлагают минималистичный интерфейс без инструментов анализа (Google Keep, Apple Notes), либо требуют глубокого погружения в настройку графовых связей и плагинов (Obsidian, Logseq), что создаёт высокий порог входа.

Концепция «второго мозга» (Second Brain), популяризированная в исследованиях по управлению персональными знаниями, предлагает подход к систематизации информации через создание взаимосвязанных структур

					ДП.09.02.07.01.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

знаний [3, 4]. Однако большинство существующих инструментов либо требуют сложной настройки, либо не обеспечивают интеграцию трекера задач напрямую в заметки.

Кроме того, большинство платформ не интегрируют трекер задач напрямую в структуру знаний, вынуждая пользователя дублировать информацию в календарях или внешних task-менеджерах. Это создаёт разрыв между генерацией идеи и её реализацией, что негативно сказывается на эффективности работы.

Таким образом, выявлена потребность в веб-приложении, объединяющем создание заметок с поддержкой разметки, управление задачами со статусами и дедлайнами, а также интерактивную визуализацию ассоциативных связей в единой среде. Такой инструмент позволит снизить когнитивную нагрузку, ускорить поиск релевантной информации и повысить эффективность планирования для студентов, исследователей и специалистов, работающих с большими объёмами данных.

1.2 Предпроектное обследование объекта

1.2.1 Анализ целевой аудитории

Целевой аудиторией системы «Romni» являются специалисты, студенты и исследователи, работающие с большими объемами информации. Проведенный анализ позволяет выделить две ключевые группы пользователей:

1. Студенты и аспиранты.

- Потребности: Ведение конспектов, подготовка к экзаменам, написание дипломных работ, трекинг дедлайнов.
- Проблемы: Потеря материалов из разных семестров, сложность в связывании тем лекций друг с другом.

2. Специалисты (фрилансеры, IT-разработчики, аналитики).

- Потребности: Фиксация рабочих гипотез, ведение базы знаний по проектам, планирование задач.

- Проблемы: Информация размазана между мессенджерами, блокнотами и таск-трекерами; отсутствие единой картины по проекту.

3. Обычные пользователи

- Потребности: Ведение личных заметок, хранение идей или важной информации, планирование повседневных дел и личных целей.

- Проблемы: Записи теряются в заметках телефона или разрозненных файлах, нет удобного способа структурировать личную информацию и отслеживать выполнение задач

Портрет потенциального пользователя:

Активный пользователь, ценящий структурированность данных. Предпочитает гибкие инструменты статическим. Готов тратить время на настройку системы в начале, чтобы сэкономить его в будущем.

1.2.2 Анализ текущего процесса управления информацией

На данный момент задача управления знаниями решается фрагментарно. Типовой процесс работы пользователя без специализированного ПО выглядит следующим образом:

1. Фиксация идеи: Пользователь создает заметку в любом доступном средстве (текстовый файл, мессенджер, стикер). Часто контекст (дата, источник, связь с другой идеей) теряется.

2. Планирование задачи: Если заметка подразумевает действие, пользователь вынужден открывать календарь или другой таск-трекер и создавать дублирующую запись.

3. Поиск информации: Спустя время, когда нужно найти связанную информацию, пользователь использует полнотекстовый поиск, который часто выдает множество нерелевантных результатов. Логическая связь между файлами восстанавливается только в памяти пользователя.

Выявленные недостатки текущего подхода:

- Высокая когнитивная нагрузка (необходимость помнить контекст).
- Дублирование данных в разных сервисах.

- Отсутствие визуального представления структуры знаний.

1.3 Анализ существующих решений

На рынке программного обеспечения для управления персональными знаниями и задачами представлено множество решений. Для обоснования актуальности разработки системы «Pomni» был проведен сравнительный анализ трёх наиболее популярных аналогов: **Obsidian**, **Notion** и **Google Keep**.

Критерии сравнения были выбраны исходя из функциональных требований к разрабатываемой системе:

1. Наличие визуального графа связей (ключевая особенность РКМ-систем нового поколения).
2. Встроенный таск-трекинг (статусы, дедлайны, приоритеты).
3. Поддержка Markdown-разметки для гибкого форматирования.
4. Облачная синхронизация между устройствами.
5. Система авторизации и защиты персональных данных.

Результаты сравнительного анализа представлены в таблице 1.

Таблица 1 – Сравнительная характеристика аналогов системы «Pomni»

Критерий	Obsidian	Notion	Google Keep	Pomni (проект)
Визуальный граф связей	+ (требуется плагины)	–	–	+ (встроенный)
Таск-трекинг (статусы, дедлайны)	– (через плагины)	+	+(базовый)	+(интегрированный)
Markdown-редактор	+	+	–	+
Облачная синхронизация	+/- (платно / через сторонние сервисы)	+	+	+
Личный кабинет и безопасность	+	+	+	+

Порог входа (сложность освоения)	Высокий	Средний	Низкий	Низкий
Стоимость базовой версии	Бесплатно	Бесплатно	Бесплатно	Бесплатно

Анализ аналогов

1. Obsidian

- Преимущества: Мощная работа с локальными Markdown-файлами, гибкая система связей между заметками, большое сообщество и библиотека плагинов.
- Недостатки: Высокий порог входа — для настройки графа и таск-трекинга требуются сторонние плагины и технические знания. Отсутствие встроенной облачной синхронизации в бесплатной версии. Нет нативной интеграции задач с дедлайнами в структуру графа.
- Вывод: Решение перегружено для пользователя, который хочет быстро начать работать, а не настраивать систему.

2. Notion

- Преимущества: Гибкая система баз данных, канбан-доски, календари, отличная командная работа, облачная синхронизация «из коробки».
- Недостатки: Отсутствие визуального графа связей между заметками — ключевой функции для ассоциативного мышления. Интерфейс может быть избыточным для личного использования. Работает только онлайн, офлайн-режим ограничен.
- Вывод: Отличный инструмент для проектной работы, но не подходит для визуализации личных знаний в виде графа.

3. Google Keep

- Преимущества: Максимальная простота, мгновенная синхронизация с экосистемой Google, поддержка меток и чек-листов.

- Недостатки: Отсутствие иерархической структуры, связей между заметками, продвинутого форматирования (Markdown). Нет визуализации знаний. Функционал ограничен базовыми заметками.

- Вывод: Подходит для быстрых записей, но не для системного управления знаниями.

Проведенный анализ показал, что ни одно из существующих решений не объединяет в себе простоту использования, визуальный граф связей и интегрированный таск-трекинг в единой среде.

Таблица 2 – Соответствие проблем аналогов и решений в системе «Pomni»

Проблема аналогов	Решение в «Pomni»
Obsidian сложен в настройке	Интуитивный интерфейс «из коробки», граф и задачи работают сразу
Notion не имеет графа знаний	Встроенный интерактивный граф, отображающий связи между заметками и задачами
Google Keep слишком примитивен	Поддержка Markdown, иерархия, теги и расширенное форматирование
Разрыв между заметкой и задачей	Задача может быть создана напрямую из заметки с привязкой к дедлайну и статусу

Уникальные преимущества проекта:

1. **Единая среда:** Заметка и задача — части одного объекта. Пользователь не переключается между приложениями.

2. **Визуализация контекста:** Граф показывает не только связи между идеями, но и привязанные к ним задачи с дедлайнами.

3. **Низкий порог входа:** Минималистичный интерфейс, не требующий изучения плагинов или сложной настройки.

4. **Веб-доступность:** Работа в браузере без установки, синхронизация между устройствами.

Существующие решения покрывают либо задачу ведения заметок, либо планирования, но не обеспечивают их глубокую интеграцию с визуализацией связей. Разработка системы «Romni» обоснована потребностью в инструменте, который объединяет преимущества аналогов, устраняя их ключевые недостатки.

Глава 2. Проектная часть

2.1 Описание процесса разработки модели предмета (объекта) исследования

Для детального проектирования системы «Romni» был выполнен декомпозиционный анализ основного процесса управления знаниями. Моделирование выполнялось в нотации IDEF0, что позволяет чётко разграничить функциональные блоки, потоки данных и управляющие воздействия.

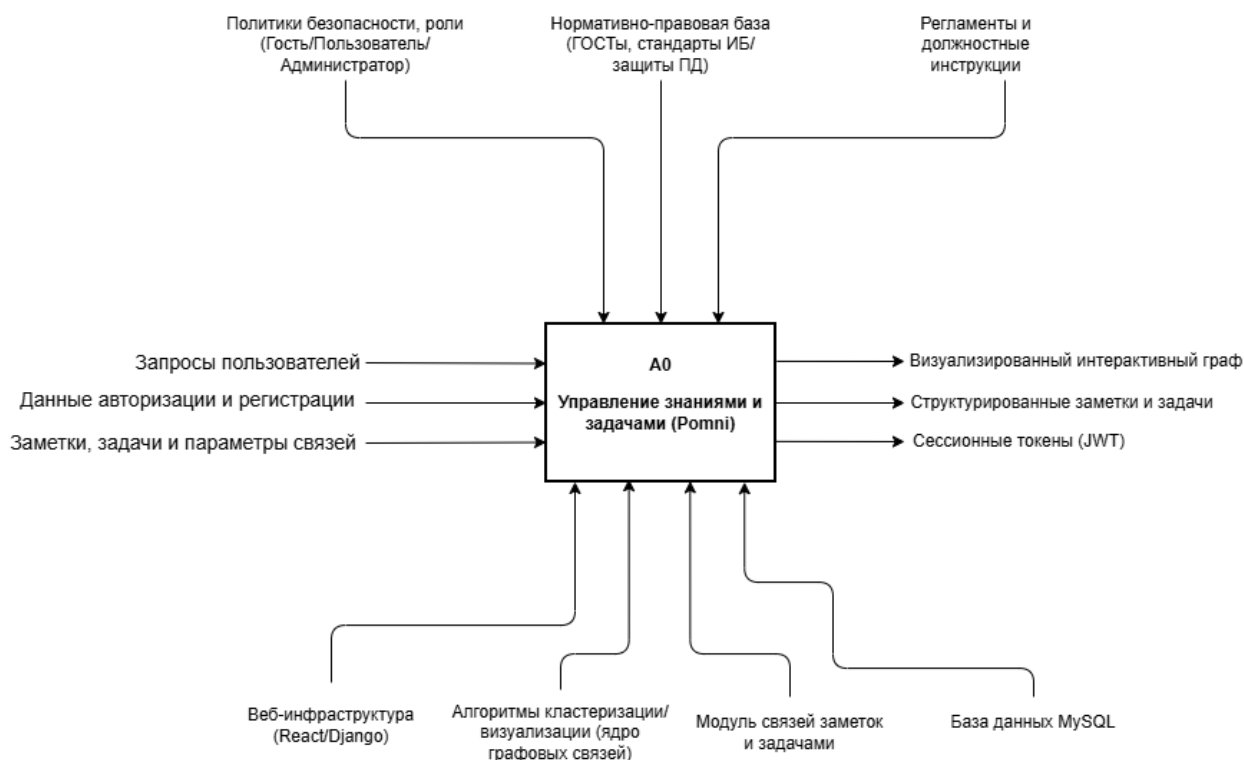


Рисунок 1 – Диаграмма IDEF0 (A0) «Управление знаниями и задачами»

На основе диаграммы выделены следующие элементы процесса:

1. Входы (Данные, поступающие на обработку):

- **Личные заметки и задачи:** Текстовый контент, создаваемый пользователем.
- **Запросы пользователей:** Команды на поиск, редактирование или удаление данных.
- **Данные авторизации:** Логины и пароли для идентификации личности.
- **Данные внешних сервисов:** Импорт информации из облачных хранилищ или календарей.

2. Управление (Ограничения и нормативы):

- **Нормативно-правовая база:** Федеральные законы (в т.ч. 152-ФЗ «О персональных данных»), регламентирующие хранение данных.
- **Политики безопасности:** Роли доступа (Администратор, Пользователь, Гость), метрики безопасности.
- **Регламенты:** Должностные инструкции по администрированию системы.

3. Механизмы (Ресурсы для выполнения):

- **Веб-инфраструктура:** Сервер VPS, СУБД MySQL, фреймворки React и Node.js.
- **Алгоритмы:** Ядро графовых связей, модули кластеризации и визуализации данных.
- **Модуль резервного копирования:** Обеспечение сохранности данных.

4. Выходы (Результат процесса):

- **Визуализированный интерактивный граф:** Наглядное представление связей между сущностями.
- **Структурированные данные:** Отформатированные заметки с тегами и статусами.
- **Экспорт данных:** Файлы для переноса в другие среды.

					ДП.09.02.07.01.000 ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подп.	Дата		

- **Сессионные токены (JWT):** Маркеры доступа для авторизованных пользователей.

Анализ показал, что существующие способы организации данных (текстовые файлы, линейные списки) не позволяют эффективно выстраивать ассоциативные связи. Разрабатываемая система «Romni» позволит перевести процесс из хаотичного накопления файлов в структурированное управление базой знаний с возможностью визуального анализа связей

На основе контекстной диаграммы А0 была построена диаграмма первого уровня декомпозиции, раскрывающая внутреннюю структуру системы.

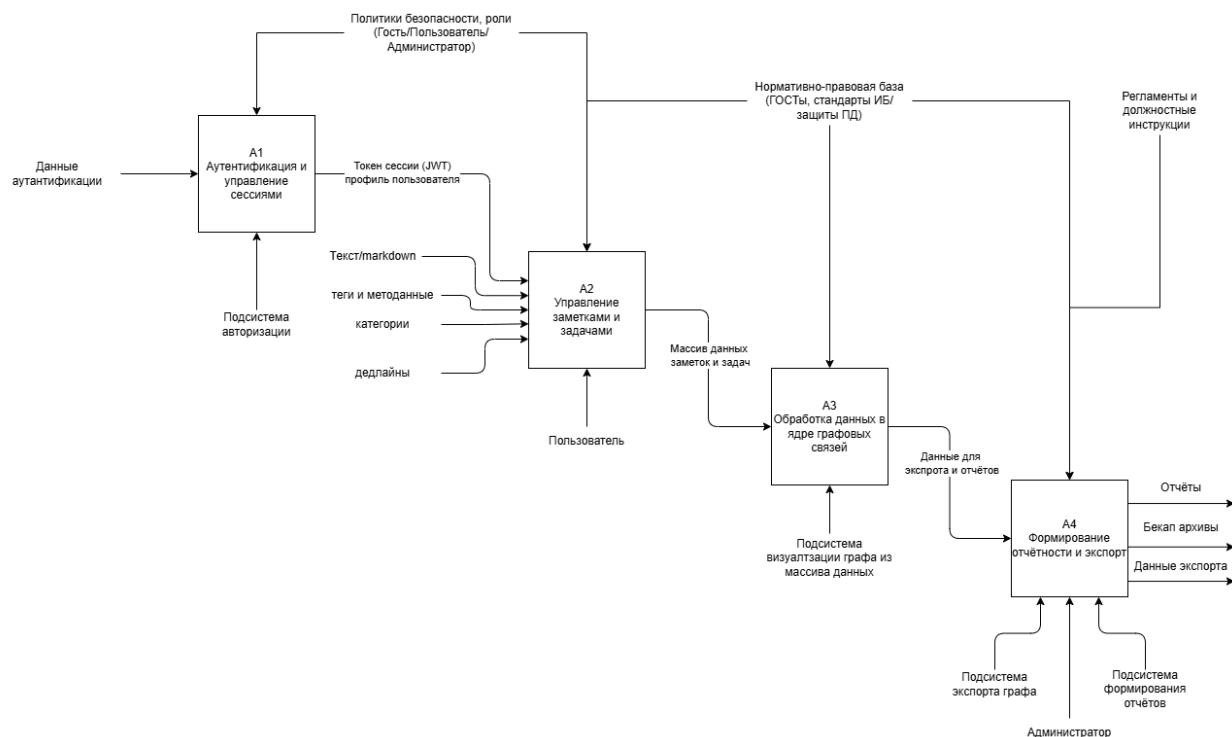


Рисунок 2 – Основные процессы

Система разделена на четыре ключевых функциональных блока, каждый из которых отвечает за определенную группу задач. Рассмотрим их подробнее:

Блок А1: Аутентификация и управление сессиями

Данный блок является точкой входа в систему. Он отвечает за безопасность и идентификацию пользователей.

- Входные данные: Учетные данные пользователя (логин/пароль), данные для регистрации.
- Функции:
 - Валидация вводимых данных на соответствие формату.
 - Проверка хэша пароля в базе данных.
 - Генерация и выдача JWT-токена (JSON Web Token) при успешном входе.
 - Управление жизненным циклом сессии (продление, истечение срока, выход из системы).
- Выходные данные: Токен доступа, профиль пользователя (роль, настройки).
- Управление: Политики безопасности, регламенты хранения паролей.

Блок А2: Управление заметками и задачами

Центральный модуль системы, обеспечивающий создание и редактирование контента.

- Входные данные: Текст заметки (Markdown), теги, категории, параметры задач (дедлайн, статус, приоритет).
- Функции:
 - Создание, чтение, обновление и удаление (CRUD) заметок.
 - Парсинг Markdown-разметки для отображения форматированного текста.
 - Привязка задач к конкретным заметкам или категориям.
 - Индексация контента для быстрого поиска.
- Выходные данные: Массив данных заметок и задач, готовый для сохранения и визуализации.

					ДП.09.02.07.01.000 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подп.	Дата		

- Механизмы: Редактор текста, модуль тегирования.

Блок А3: Обработка данных в ядре графовых связей

Уникальный модуль системы, отвечающий за аналитику и визуализацию связей.

- Входные данные: Массив данных из блока А2 (заметки с тегами и ссылками).
- Функции:
 - Анализ метаданных заметок для выявления связей (общие теги, перекрестные ссылки).
 - Построение матрицы смежности графа.
 - Расчет координат узлов для оптимального отображения (алгоритмы force-directed graph).
- Выходные данные: Структурированные данные для отрисовки графа (узлы и ребра).
- Механизмы: Алгоритмы кластеризации и визуализации (например, библиотека D3.js или Sigma.js).

Блок А4: Формирование отчётности и экспорт

Модуль, обеспечивающий выгрузку данных и администрирование.

- Входные данные: Запросы на экспорт, данные для резервного копирования.
- Функции:
 - Генерация файлов экспорта (JSON, Markdown, PDF) по запросу пользователя.
 - Создание резервных копий базы данных (Backup) по расписанию.
 - Формирование административных отчетов (активность пользователей, объем данных).
- Выходные данные: Файлы экспорта, архивы БД, отчеты.
- Управление: Регламенты резервного копирования, настройки формата экспорта.

Взаимодействие модулей

Процессы в системе тесно связаны между собой:

1. Пользователь проходит аутентификацию в блоке А1, получая доступ к интерфейсу.
2. Через интерфейс пользователь вводит данные в блок А2, создавая заметки и задачи.
3. Данные из блока А2 автоматически передаются в блок А3, где ядро графа обрабатывает связи и формирует визуальную модель.
4. При необходимости пользователь или администратор инициирует процесс в блоке А4 для сохранения результатов работы или выгрузки данных.

Такое разделение обеспечивает модульность архитектуры: изменения в алгоритмах визуализации (А3) не затрагивают логику работы с текстом (А2), а обновление системы безопасности (А1) не влияет на функции экспорта (А4).

2.2 Требования к информационной системе

На основе анализа предметной области и моделирования бизнес-процессов были сформулированы требования к разрабатываемой системе «Romni». Требования разделены на две группы: функциональные (описывают возможности системы) и нефункциональные (определяют технические ограничения и характеристики).

2.2.1 Функциональные требования

Функциональные требования описывают действия, которые система должна выполнять для удовлетворения потребностей пользователей.

Таблица 3 – Функциональные требования к системе «Pomni»

№	Требование	Описание
1	Регистрация и аутентификация	Система должна предоставлять возможность регистрации нового пользователя, входа в систему по логину и паролю, а также безопасного выхода из сессии.
2	Управление заметками	Система должна позволять создавать, редактировать, просматривать и удалять заметки с поддержкой Markdown-разметки.
3	Организация структуры	Система должна предоставлять возможность группировать заметки по категориям и присваивать им теги для быстрой фильтрации.
4	Управление задачами	Система должна позволять создавать задачи, привязанные к заметкам, устанавливать статусы (Новая, В работе, Готово), приоритеты и дедлайны.
5	Визуализация графа	Система должна автоматически строить интерактивный граф связей на основе тегов и перекрёстных ссылок между заметками.
6	Поиск и фильтрация	Система должна обеспечивать полнотекстовый поиск по содержимому заметок и фильтрацию по тегам, категориям и статусам задач.
7	Экспорт данных	Система должна позволять экспортировать заметки и задачи в форматах JSON, Markdown и PDF.
8	Резервное копирование	Система должна автоматически создавать резервные копии данных пользователя с возможностью восстановления.

2.2.2 Нефункциональные требования

Таблица 4 – Нефункциональные требования к системе «Romni»

№	Требование	Обоснование
1	Веб-архитектура	Система должна быть реализована как веб-приложение, доступное через браузер без установки клиентского ПО. Это обеспечивает кроссплатформенность и упрощает обновление.
2	Адаптивный интерфейс	Интерфейс должен корректно отображаться на экранах с разрешением от 320px (мобильные устройства) до 1920px (десктопы).
3	Время отклика	Время загрузки страницы и отклика на действия пользователя не должно превышать 2 секунд при стандартном интернет-соединении.
4	Безопасность данных	Пароли пользователей должны храниться в хэшированном виде (алгоритм bcrypt). Передача данных должна осуществляться по защищённому протоколу HTTPS.
5	Соответствие 152-ФЗ	Система должна обеспечивать хранение персональных данных пользователей на территории РФ и предоставлять возможность их удаления по запросу.
6	Масштабируемость	Архитектура системы должна позволять увеличивать количество одновременных пользователей без существенной модификации кода (горизонтальное масштабирование).

2.3 Проектирование базы данных

2.3.1 Выбор системы управления базами данных

Для хранения данных системы «Romni» выбрана реляционная СУБД MySQL 8.0. Данный выбор обоснован следующими факторами:

- Надёжность и стабильность: MySQL — проверенное решение для веб-приложений с поддержкой транзакций ACID.

- Совместимость со стеком технологий: Полная интеграция с Django ORM (Object-Relational Mapping), что упрощает работу с данными на уровне кода.
- Производительность: Поддержка индексов, оптимизация запросов, работа с большими объёмами текста (поля longtext).
- Бесплатность и открытость: Лицензия GPL позволяет использовать СУБД без дополнительных затрат.
- Поддержка внешних ключей: Обеспечение целостности данных между связанными таблицами.

2.3.2 Концептуальная модель данных

Анализ предметной области позволил выделить ключевые сущности системы:

1. **Пользователь (User)** — владелец данных, проходит аутентификацию.
2. **Заметка (Note)** — основной объект хранения информации. Поддерживает иерархию (заметка может быть папкой).
3. **Задача (Task)** — действие, привязанное к заметке.
4. **Статус задачи (Status)** — справочник состояний задачи (например, «Новая», «В работе», «Выполнено»).
5. **Связи (Links)** — таблица для оптимизации запросов при построении графа и дерева папок.

Особенность проектирования: В системе реализован гибкий подход к организации заметок. Любая заметка может стать «папкой», если у неё появляются дочерние элементы. Это определяется наличием записей в таблице связей, где данная заметка выступает в роли родителя.

2.3.3 Логическая структура базы данных

Для аутентификации и хранения данных пользователей используется стандартная модель Django auth_user, расширенная дополнительным полем phone

Таблица 5 – auth_user

Поле	Тип
id	Int
username	Varchar(150)
password	Varchar(128)
email	Varchar(254)
phone	Varchar(20)

Таблица 6 – notes

Хранит текст заметок и их иерархическую структуру. Поле folder_id ссылается на саму себя, создавая рекурсивную связь (дерево).

Поле	Тип
id	Bigint
title	Varchar(255)
text	Longtext
folder_id	Bigint
user_id	Int
created_at	Datetime(6)
updated_at	Datetime(6)

Таблица 7 – links

Поле	Тип
id	int
folder_id	Bigint
note_id	Bigint

Таблица 8 – statuses

Поле	Тип
id	int
name	Varchar(20)

Таблица 9 – tasks

Поле	Тип
id	Bigint
title	Varchar
description	Text
note_id	Bigint
user_id	Int
status	Int
priority	Int
due_date	Datetime
completed_at	Datetime
created_at	Datetime

2.3.4 Схема БД

На рисунке 3 представлена итоговая ER-диаграмма, отображающая связи между таблицами.

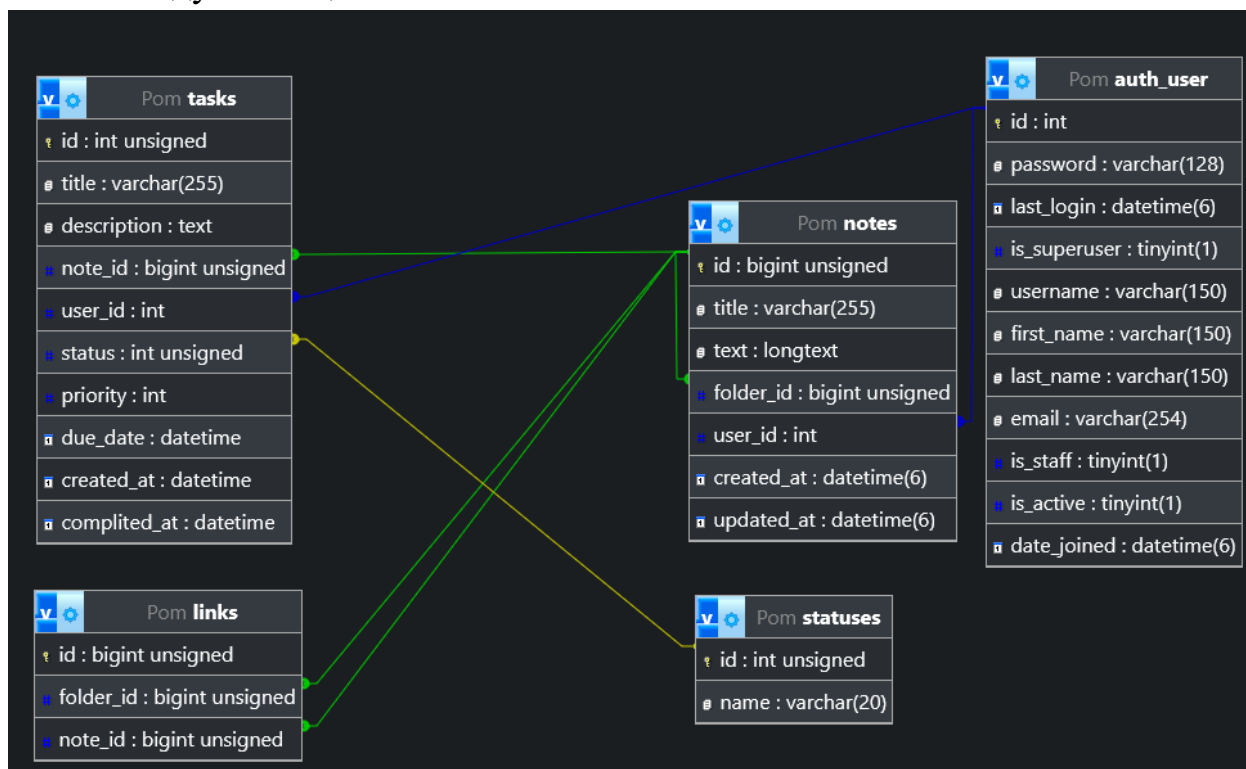


Рисунок 3 – Примерная схема БД проекта

Описание связей:

1. **User — Notes / Tasks:** Связь «один ко многим». Один пользователь может создавать множество заметок и задач. При удалении пользователя данные удаляются каскадно.
2. **Notes — Links:** Связь для построения иерархии. Позволяет одной заметке быть родительской для других.
3. **Tasks — Statuses:** Связь «многие к одному». Многие задачи могут иметь один и тот же статус.
4. **Tasks — Notes:** Задача всегда привязана к конкретной заметке (note_id).

Спроектированная схема базы данных является нормализованной и оптимизированной для работы веб-приложения. Использование рекурсивных связей в таблице notes и выносной таблицы links позволяет эффективно реализовывать функционал «Графа знаний» и файловую структуру одновременно.

2.4 Построение функционально-алгоритмической структуры

2.4.1 Алгоритм аутентификации и авторизации пользователя

Процесс входа в систему является критически важным для обеспечения безопасности персональных данных. Алгоритм реализован на стороне сервера с использованием хэширования паролей.

Последовательность действий:

1. Пользователь вводит логин и пароль в клиентском интерфейсе.
2. Клиент отправляет POST-запрос на сервер (API endpoint).
3. Сервер проверяет наличие пользователя с указанным логином в базе данных.
4. Если пользователь не найден: Возвращается ошибка 401 «Неверные данные».
5. Если пользователь найден: Сервер берет введенный пароль, применяет к нему алгоритм хэширования (соление + bcrypt) и сравнивает полученный хэш с хэшем, хранящимся в БД.
6. Если хэши не совпадают: Возвращается ошибка 401.
7. Если хэши совпали:
 - Генерируется сессионный токен (или JWT).
 - В базу данных записывается время последнего входа (last_login).
 - Токен отправляется клиенту.
8. Клиент сохраняет токен и предоставляет доступ к личному кабинету.

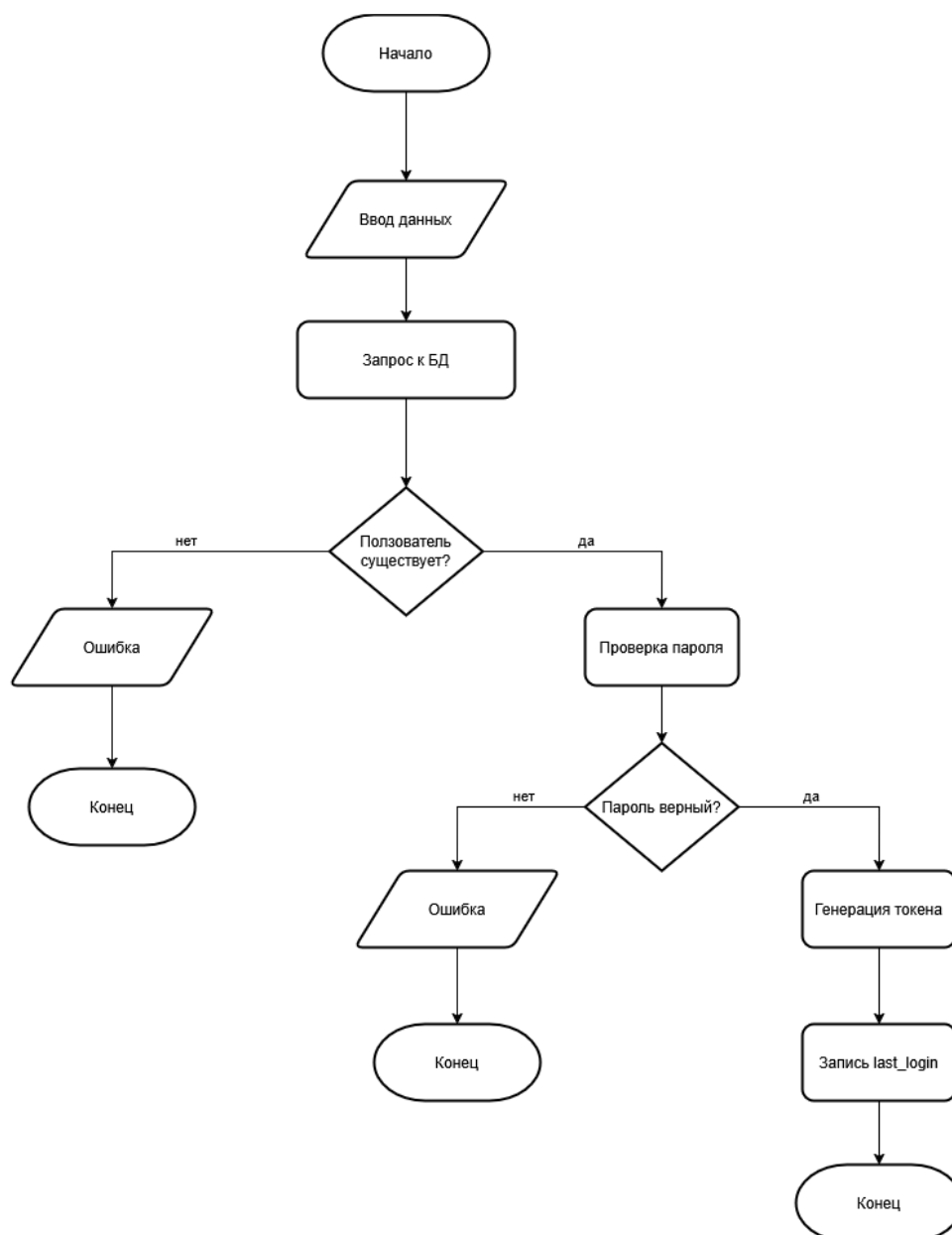


Рисунок 4 – Блок схема алгоритма аутентификации

2.4.2 Алгоритм создания и сохранения заметки

Данный алгоритм описывает процесс обработки контента, создаваемого пользователем. Особенностью системы является динамическое определение статуса заметки (файл или папка) на основе связей.

Последовательность действий:

1. Пользователь заполняет форму создания заметки (Заголовок, Текст, Родительская папка).
2. Сервер валидирует входящие данные (проверка на пустоту, длину строк).
3. Сервер создает запись в таблице Pom notes.

4. Обработка иерархии:

- Если указан folder_id (родительская папка), создается запись в связующей таблице Pom links (связка folder_id -> note_id).
- Если заметка создается в корне, folder_id остается NULL.

5. Система проверяет, появляются ли у текущей заметки дочерние элементы.

- Логика: Если в таблице Pom links есть записи, где текущая заметка выступает в роли folder_id, интерфейс помечает её как «Папку».

6. Данные сохраняются в БД.

7. Клиент получает подтверждение успеха и обновляет список заметок.

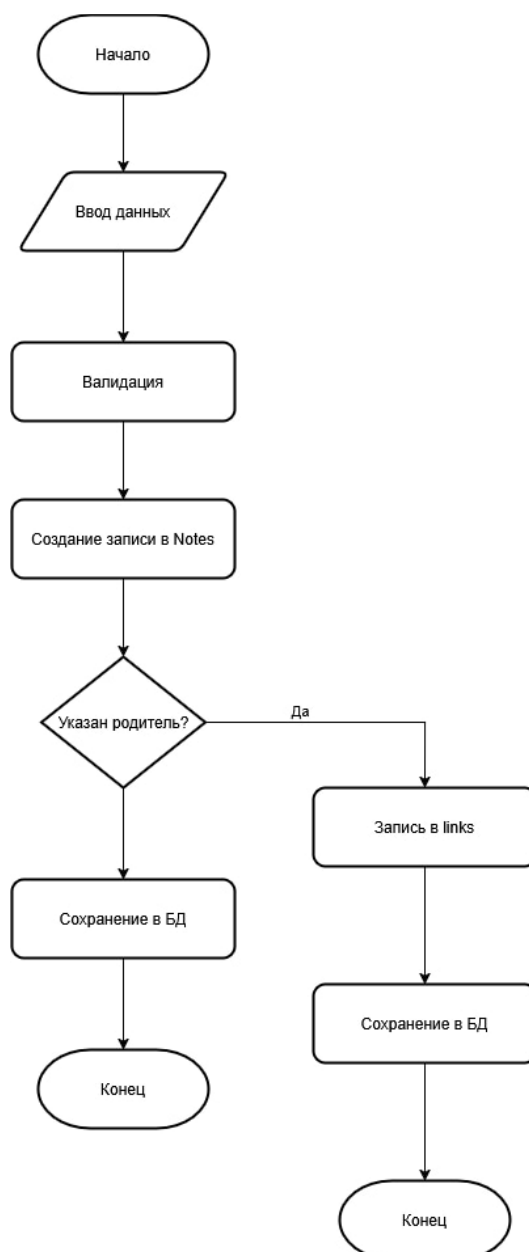


Рисунок 5 – Блок схема алгоритм создания и сохранения заметки

2.5 Проектирование пользовательского интерфейса

Для разработки приложения было решено использовать тёмные тона и светло розовые акценты для создания уютного дизайна. Для заголовков был выбран шрифт Home Video.

Основные цвета:

#FFBEEF – Светло розовый цвет акцентов

#FBFF87 – Светло жёлтый цвет акцентов

#151515 – Цвет фона приложения

#FFFFFF – Цвет текста приложения

#252525 – Цвет модальных окон и элементов на странице

#AAAAAA – Цвет обводки элементов

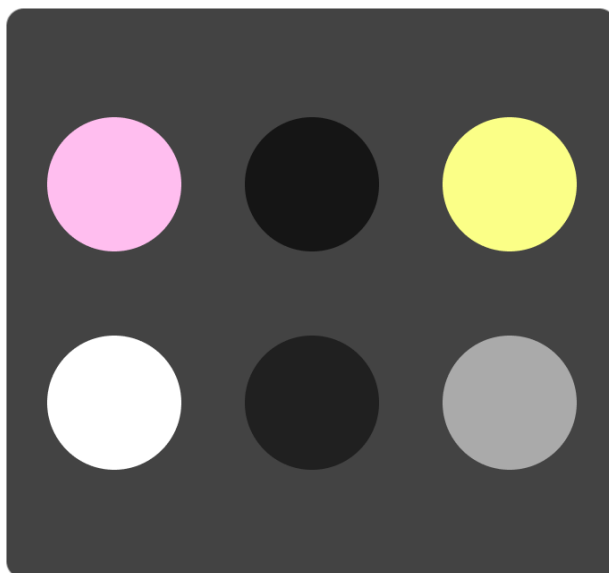


Рисунок 6 – Основные цвета приложения



Рисунок 7 – Логотип приложения

Страницы приложения:

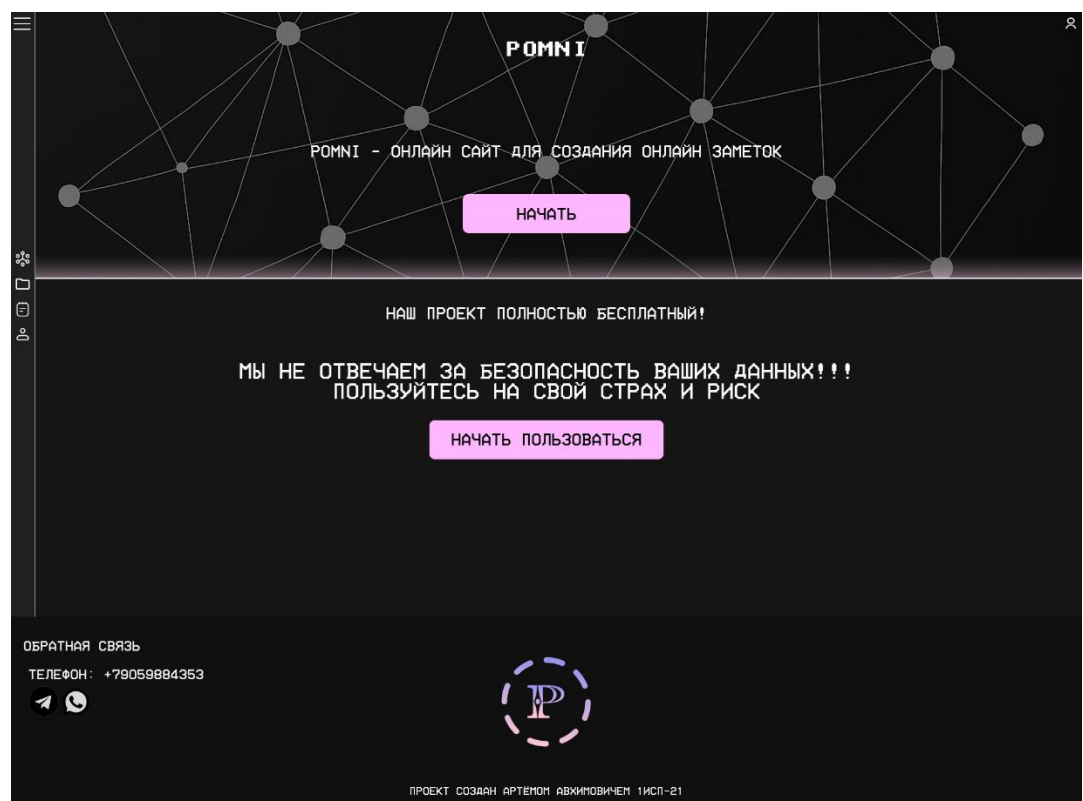


Рисунок 8 – Главная страница

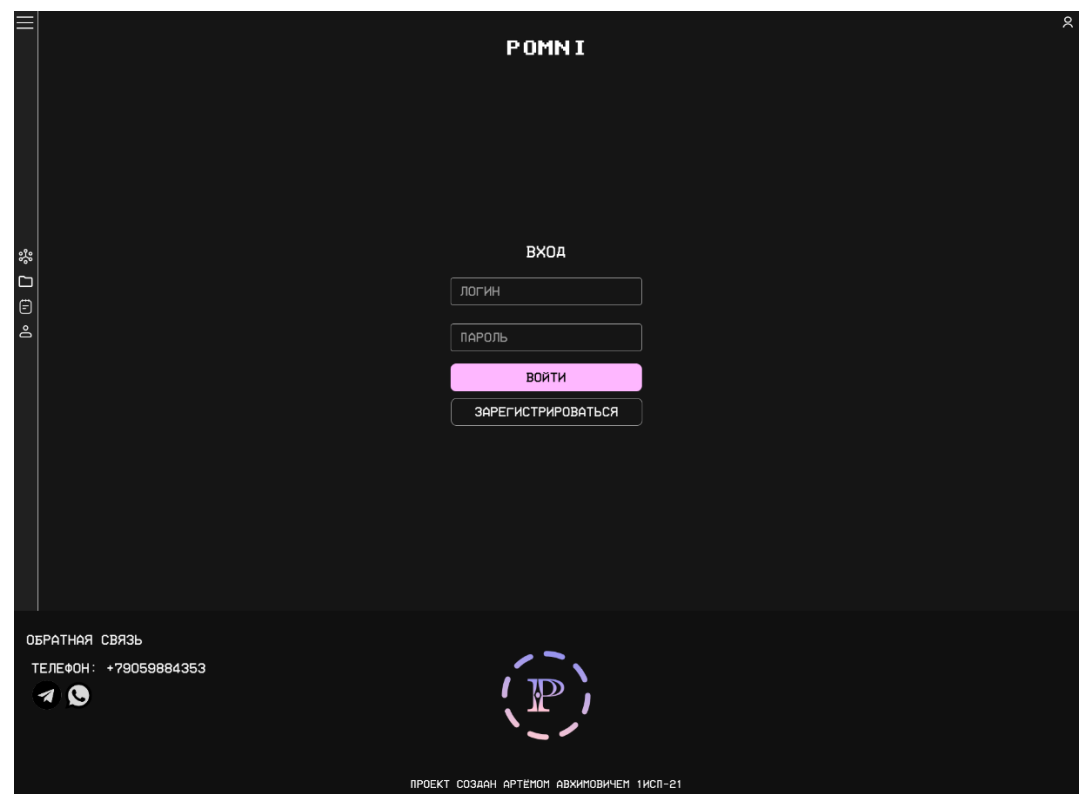


Рисунок 9 – Страница авторизации

Рисунок 10 – Страница регистрации

Рисунок 11 – Страница личного кабинета

					ДП.09.02.07.01.000 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подп.	Дата		

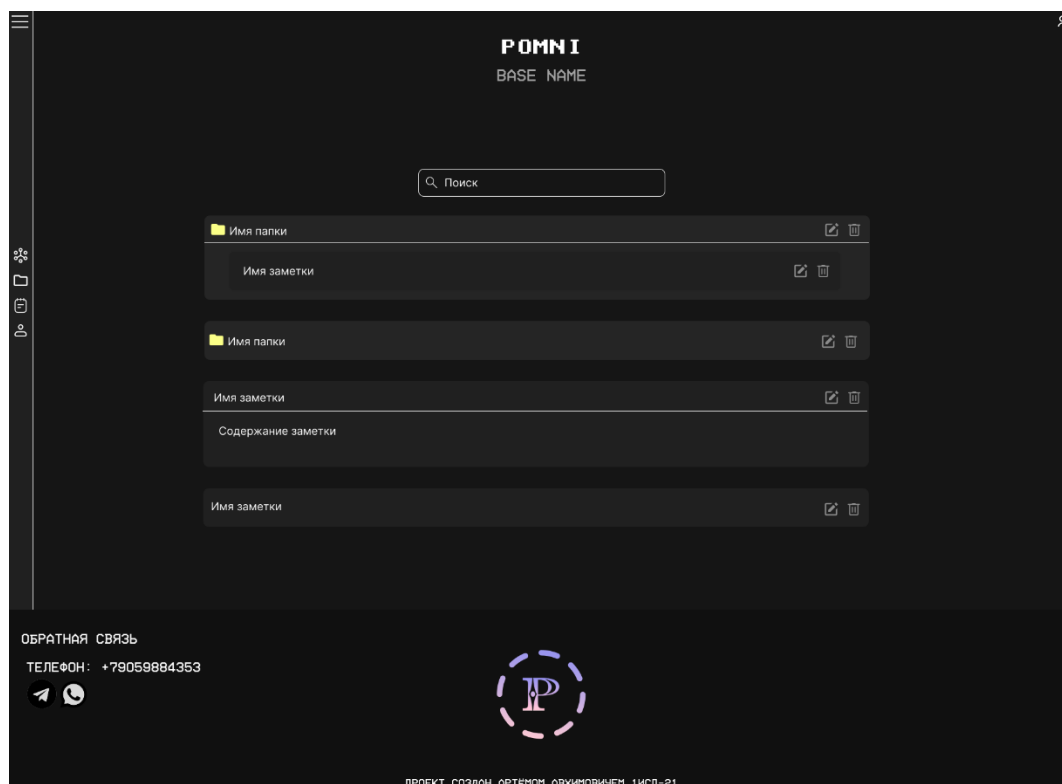


Рисунок 12 – Страница файлового вида

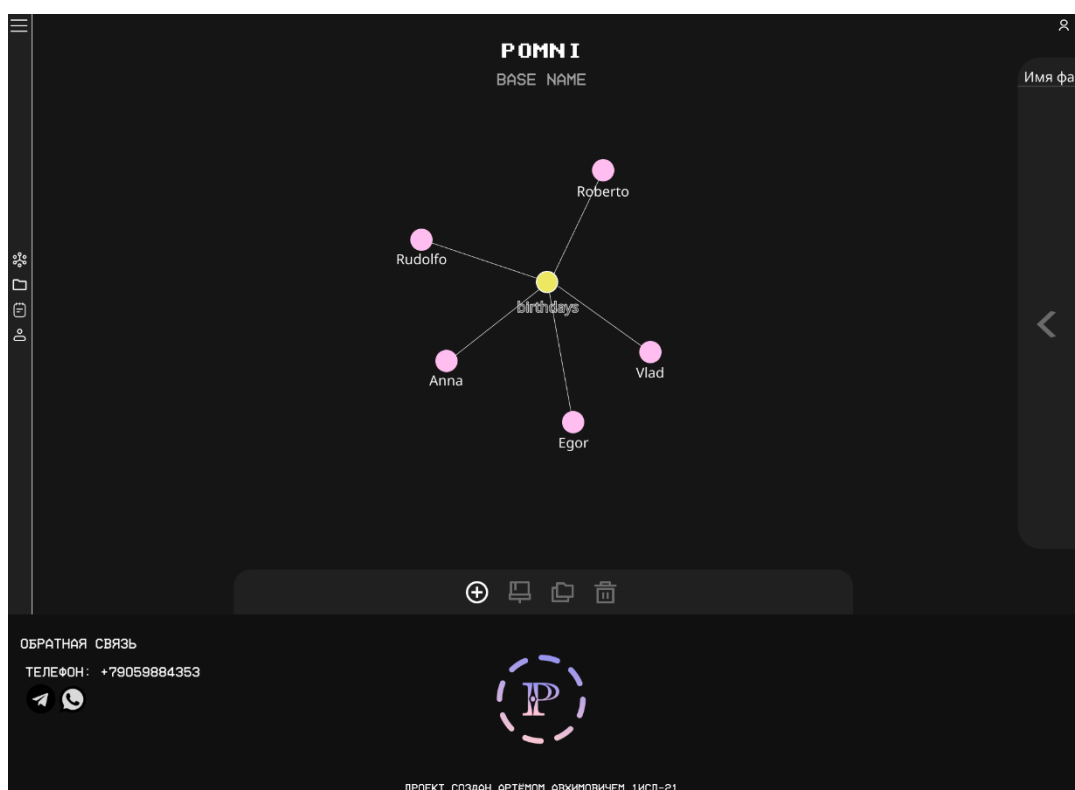


Рисунок 13 – Страница графа

					ДП.09.02.07.01.000 ПЗ	Лист
						29
Изм.	Лист	№ докум.	Подп.	Дата		

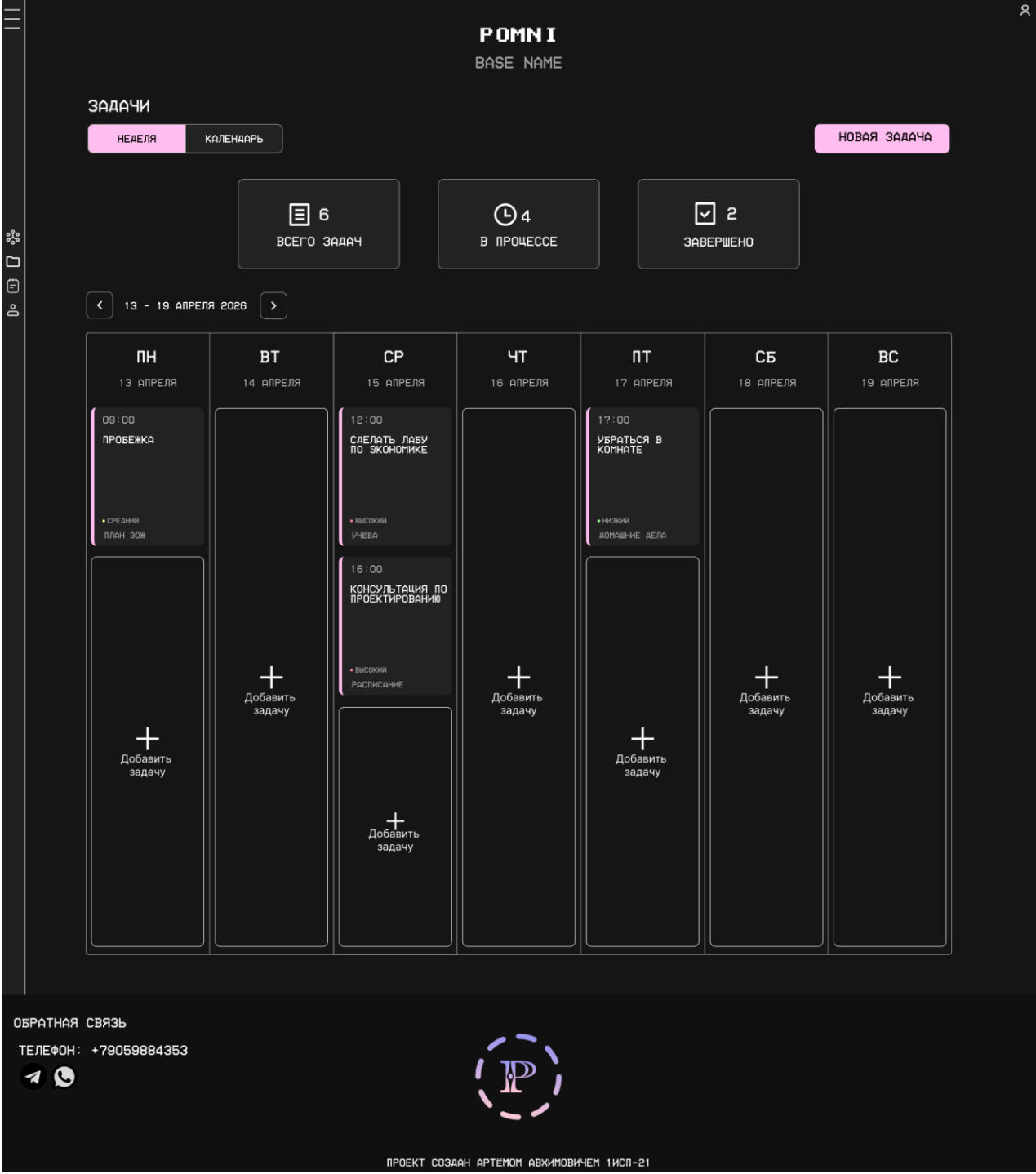


Рисунок 14 – Страница задач (недельный вид)

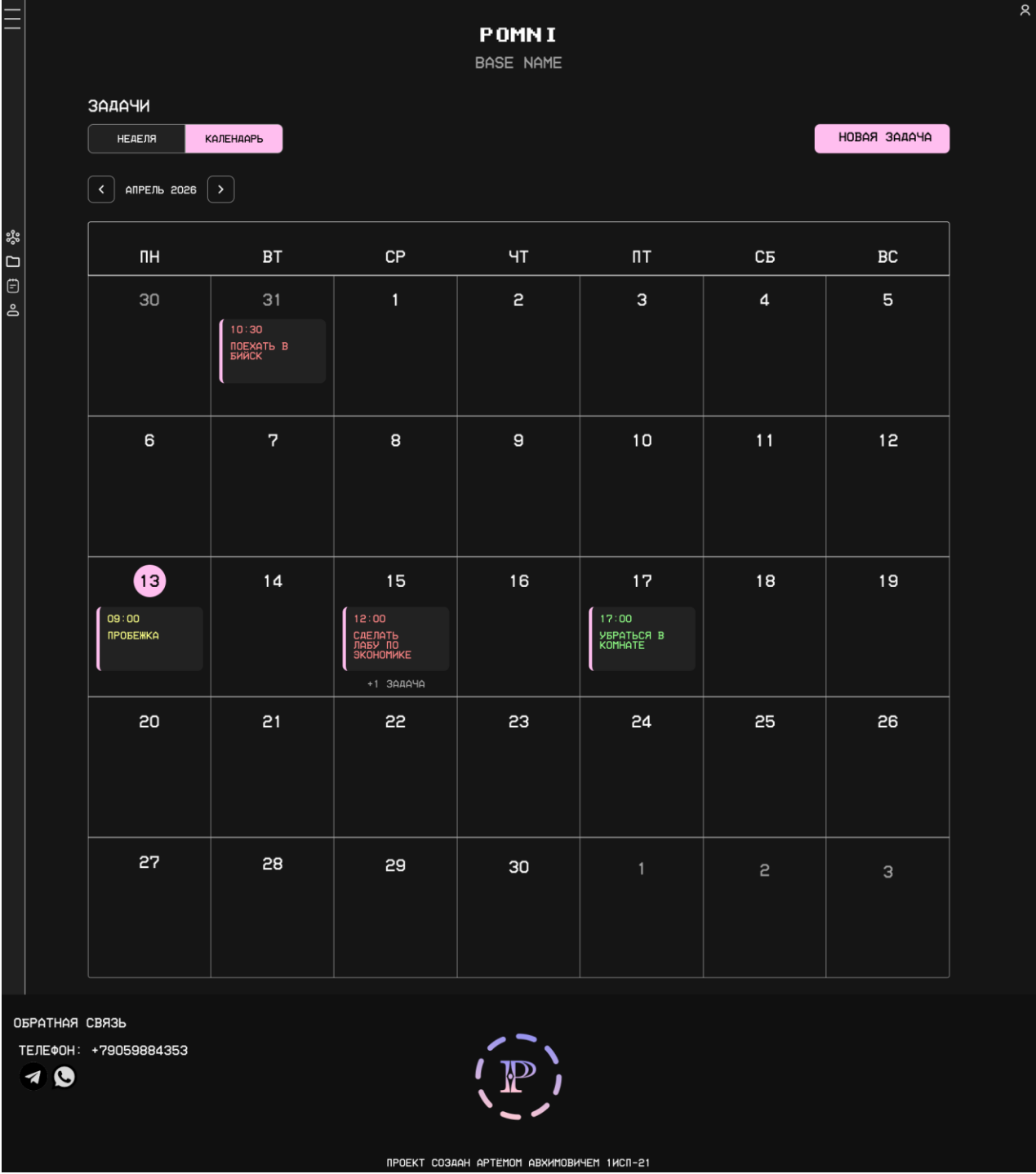


Рисунок 15 – Страница задач (календарный вид)

Модальные окна:

×

СОЗДАНИЕ ЗАМЕТКИ

НАЗВАНИЕ

ВВЕДИТЕ НАЗВАНИЕ ЗАМЕТКИ

ОПИСАНИЕ

ВВЕДИТЕ ОПИСАНИЕ ЗАМЕТКИ

ЗАМЕТКА

ВЫБЕРИТЕ ПАПКУ

ОТМЕНА

СОЗДАТЬ

СОЗДАТЬ ПАПКУ

Рисунок 16 – Окно создания заметки

×

СОЗДАНИЕ ПАПКИ

НАЗВАНИЕ

ВВЕДИТЕ НАЗВАНИЕ ПАПКИ

ОТМЕНА

СОЗДАТЬ

СОЗДАТЬ ЗАМЕТКУ

Рисунок 17 – Окно создания папки

×

СОЗДАТЬ ЗАДАЧУ

НАЗВАНИЕ

ВВЕДИТЕ НАЗВАНИЕ ЗАДАЧИ

ОПИСАНИЕ

ВВЕДИТЕ ОПИСАНИЕ ЗАДАЧИ

ДАТА

ДД.ММ.ГГГГ

📅

ВРЕМЯ

00:00

🕒

ПРИОРИТЕТ

ВЫСОКИЙ

СРЕДНИЙ

НИЗКИЙ

СТАТУС

ПЛАНИРУЮ

▼

ЗАМЕТКА

ВЫБИРИТЕ ЗАМЕТКУ (НЕОБЯЗАТЕЛЬНО) ▼

ОТМЕНА

СОЗДАТЬ

Рисунок 18 – Окно создания задачи

2.6 Выбор стека технологий

Для данного проекта я буду использовать следующие технологии:

Frontend: React, HTML, CSS, JavaScript, D3.js, dnd-kit.

Backend: Python, Django, Django REST Framework.

База данных: MySQL

Деплой: Docker Compose

2.7 Обоснование стека технологий

Для разработки информационной системы «Romni» планируется использовать стек технологий, ориентированный на создание веб-приложения с разделением клиентской и серверной частей. Такой подход позволит независимо развивать пользовательский интерфейс, серверную бизнес-логику и слой хранения данных.

Клиентскую часть планируется реализовать с использованием React, HTML, CSS и JavaScript. React выбран как современная библиотека для разработки интерактивных пользовательских интерфейсов. Он позволит строить приложение на основе компонентов, повторно использовать элементы интерфейса и удобно управлять изменением состояния страниц. Для маршрутизации планируется использовать React Router DOM, а для управления состоянием и обмена данными с сервером — Redux Toolkit и RTK Query.

Для визуализации связей между заметками планируется использовать библиотеку D3.js. Ее применение обосновано необходимостью отображения графа знаний, где заметки и папки представлены как связанные элементы. Для реализации перемещения заметок между папками планируется использовать dnd-kit, предоставляющий готовые механизмы drag-and-drop.

Серверную часть планируется построить на языке Python с использованием Django и Django REST Framework. Django выбран благодаря встроенным средствам работы с моделями данных, миграциями, пользователями и административной частью. Django REST Framework

планируется использовать для создания REST API, через которое клиентская часть будет получать и изменять данные.

В качестве системы управления базами данных планируется использовать MySQL 8.0. Данная СУБД подходит для хранения структурированных данных приложения: пользователей, заметок, папок, связей между заметками и задач. MySQL поддерживает реляционную модель данных, ограничения целостности и транзакции, а также интегрируется с Django ORM.

Для авторизации пользователей планируется использовать токеновую аутентификацию Django REST Framework. После успешного входа пользователь будет получать токен, который будет применяться при последующих запросах к защищенным разделам API. Такой механизм позволит ограничить доступ к личным заметкам, папкам, задачам и профилю пользователя.

Для развертывания проекта планируется использовать Docker Compose. Он позволит запускать клиентскую часть, серверную часть и базу данных как отдельные контейнеры в единой среде, что упростит настройку проекта и сделает процесс развертывания более воспроизводимым.

Глава 3. Реализация проекта

3.1 Описание реализации

3.2 Тестирование и отладка

3.3 Внедрение

					ДП.09.02.07.01.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		35

Заключение

Список источников

1. React Documentation: сайт. – URL: <https://react.dev/reference/react> (дата обращения: 05.05.2026). – Текст: электронный.
2. Redux Toolkit Documentation: сайт. – URL: <https://redux-toolkit.js.org/> (дата обращения: 05.05.2026). – Текст: электронный.
3. React Router Documentation: сайт. – URL: <https://reactrouter.com/> (дата обращения: 05.05.2026). – Текст: электронный.
4. Feature-Sliced Design Documentation: сайт. – URL: <https://feature-sliced.design/> (дата обращения: 29.04.2026). – Текст: электронный.
5. Django Documentation, version 5.2: сайт. – URL: <https://docs.djangoproject.com/en/5.2/> (дата обращения: 05.05.2026). – Текст: электронный.
6. Django REST Framework. API Guide: сайт. – URL: <https://www.django-rest-framework.org/api-guide/> (дата обращения: 05.05.2026). – Текст: электронный.
7. Django REST Framework. Token Authentication: сайт. – URL: <https://www.django-rest-framework.org/api-guide/authentication/#tokenauthentication> (дата обращения: 05.05.2026). – Текст: электронный.
8. MySQL 8.0 Reference Manual: сайт. – URL: <https://dev.mysql.com/doc/refman/8.0/en/> (дата обращения: 05.05.2026). – Текст: электронный.
9. Docker Compose Documentation: сайт. – URL: <https://docs.docker.com/compose/> (дата обращения: 08.05.2026). – Текст: электронный.

10. JavaScript Guide // MDN Web Docs: сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript/Guide> (дата обращения: 05.05.2026). – Текст: электронный.

11. HTML // MDN Web Docs: сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 05.05.2026). – Текст: электронный.

12. CSS // MDN Web Docs: сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 05.05.2026). – Текст: электронный.

13. D3.js Documentation: сайт. – URL: <https://d3js.org/> (дата обращения: 05.05.2026). – Текст: электронный.

14. dnd kit Documentation: сайт. – URL: <https://docs.dndkit.com/> (дата обращения: 05.05.2026). – Текст: электронный.

15. Components, styles, and shared libraries // Figma: сайт. – URL: <https://www.figma.com/best-practices/components-styles-and-shared-libraries/> (дата обращения: 02.05.2026). – Текст: электронный.

16. Markdown Guide. Basic Syntax: сайт. – URL: <https://www.markdownguide.org/basic-syntax/> (дата обращения: 05.05.2026). – Текст: электронный.

17. Graph view // Obsidian Help: сайт. – URL: <https://help.obsidian.md/plugins/graph> (дата обращения: 27.04.2026). – Текст: электронный.

18. Notion: сайт. – URL: <https://www.notion.com/> (дата обращения: 27.04.2026). – Текст: электронный.

19. Google Keep: сайт. – URL: <https://workspace.google.com/products/keep/> (дата обращения: 27.04.2026). – Текст: электронный.

20. Logseq Documentation: сайт. – URL: <https://docs.logseq.com/> (дата обращения: 27.04.2026). – Текст: электронный.

21. Forte, T. Building a Second Brain: A Proven Method to Organize Your Digital Life and Unlock Your Creative Potential / T. Forte. – New York: Atria Books, 2022. – Текст: непосредственный.

22. Productivity Management Software Market Size Report, 2030 // Grand View Research: сайт. – URL: <https://www.grandviewresearch.com/industry-analysis/productivity-management-software-market> (дата обращения: 27.04.2026). – Текст: электронный.

23. Hybrid Workplace Habits & Hangups Report // Glean: сайт. – URL: <https://www.glean.com/press/hybrid-workplace-habits-hangups-report-frustrated-employees-spend-a-quarter-of-workweek-searching-for-information-needed-to-do-their-jobs> (дата обращения: 28.04.2026). – Текст: электронный.

Приложение А

Код проекта: [GitHub](#)